

PYTHON

Module 24: Array Handling & Lists

CODING

Arrays in Python

Technically, Python does not have a native data structure called 'Array'. Instead, we handle array-like behavior using **Lists**. At CodeHive, we teach Lists as the gateway to professional data management.

- **The NumPy Distinction**: For high-performance numerical computing and multidimensional arrays, professional developers transition to the **NumPy library**. However, standard Python **Lists are the built-in solution for most general-purpose grouping of data.**
-

Array-Like Properties

A List acts as an array by providing several key features:

- **Ordered Storage**: Items maintain their insertion order.
- **Indexing**: Each element has a unique numerical address starting at 0.
- **Dynamic Sizing**: Unlike arrays in languages like Java or C++, Python arrays can grow or shrink in size automatically.

Basic Operations

- ****Accessing****: Retrieve values with `array[index]` . Remember that `array[0]` is always the first item.

- ****Modifying****: Update values directly: `cars[0] = "Tesla"` .

- ****Length****: The `len()` function returns the total count of items. Pro-Tip: The highest index is always `len(array) - 1` .

Adding & Removing Elements

• **`append()`**: Adds a single item to the very end of the array.

• **`extend()`**: Merges an entire collective (like another list) into the current one.

• **`pop(index)`**: Removes and returns the item at a specific position.

• **`remove(value)`**: Searches for and deletes the *first* occurrence of a specific value found in the array.

Array/List Method Reference

Essential methods for array manipulation:

- `insert(pos, val)` : Adds an element at a specific position.
-

- `reverse()` : Flips the order of elements in-place.
-

- `sort()` : Reorders elements (alphabetically or numerically) in-place.
-

- `count(val)` : Returns how many times a value appears in the array.
-

Professional Comparison

Standard Arrays (Lists) are great for convenience and different data types in one row. However, as you advance to Module 30 (NumPy), you will find that specialized arrays offer significantly faster processing speeds and better memory efficiency for large mathematical datasets.